

# Foundations of Data Science for Biologists

## R: data types and indexing

BIO 724D

2026-SEP-01

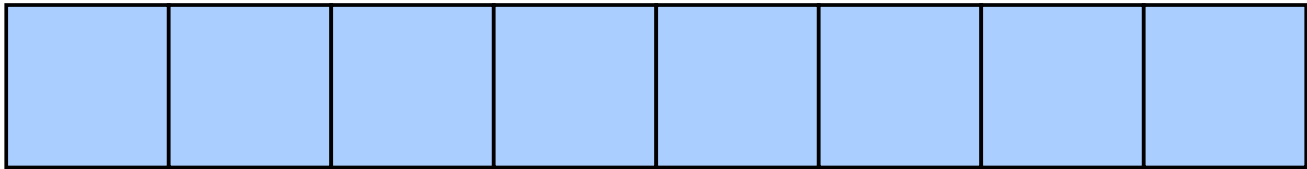
Instructors: Paul Magwene, Greg Wray

# Taxonomy of basic data types in R

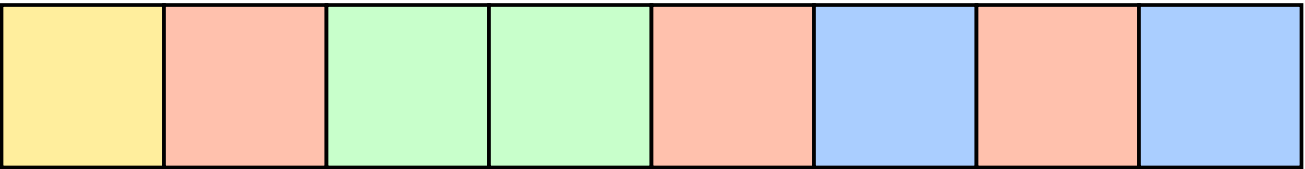
homogenous

mixed

1-dimensional

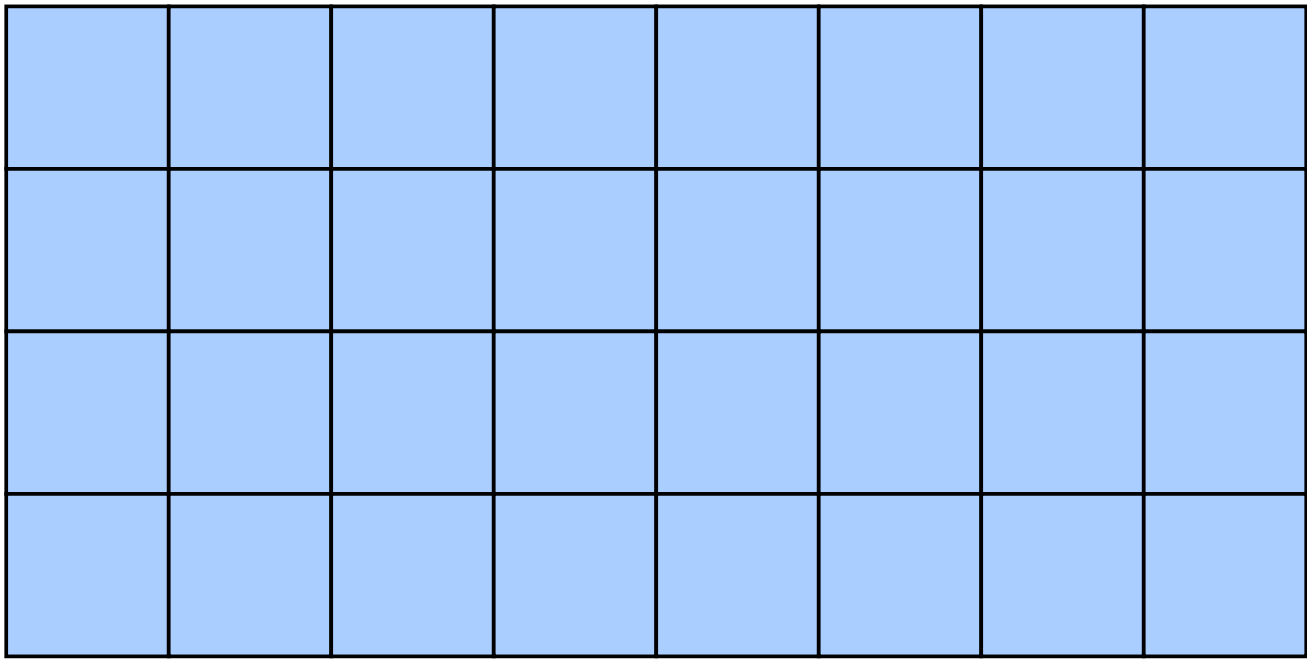


vector

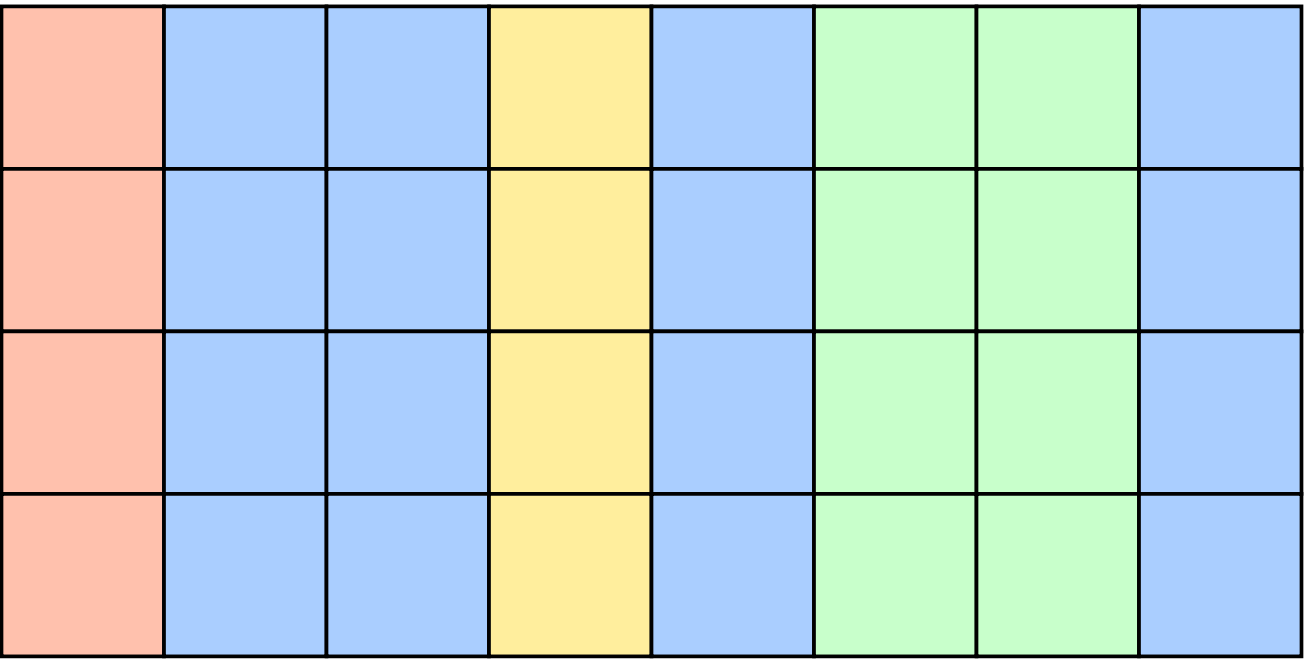


list

2-dimensional



matrix



data frame

# Data types

Computers work with 0s and 1s — but you want to work with numbers, names, dates, etc.

**Data types** instruct programs how to interpret and process different kinds of data

Basic data types in R are integer, float, character, and logical

More complex data types can be built from simpler ones, e.g., data frame

R has an extensive set of rules for each data type:

What **values** are allowed (e.g., an integer can be 42 but not "42" or 42.7)

What **operations** are allowed (e.g., division for float but not character or logical)

How to **display** data in human-readable form (e.g., 01010010 as "R" or 82)



# Storing data as objects

When you assign a value to a variable name, R stores information in two places:

A **data object** that contains:

**Data:** values, such as `-23.84` or `"Adelie"`

**Metadata:** the data type, how many values, etc.

A table of variables currently in use that contains:

**Variable names** (also called identifiers)

**Addresses** in memory

To see variables currently in use:

Refer to the Environment pane in RStudio

Use code: `ls()` (useful, for example, in an Quarto notebook)

# Retrieving data objects

When a variable name is sent to the console, R:

- Retrieves the memory address associated with that name

- Examines the metadata for the object at that address

- Interprets the data based on the metadata

The metadata tell R:

- The type of data (integer, string, logical, etc.)

- How many values are present (dimensions)

- Often, additional metadata are stored (e.g., language for strings)

# What a data frame looks like to you

penguins



| species | island    | bill_length_mm | sex    |
|---------|-----------|----------------|--------|
| Adelie  | Torgersen | 39.1           | male   |
| Adelie  | Torgersen | 39.5           | female |
| Adelie  | Biscoe    | 37.8           | female |
| Adelie  | Biscoe    | 37.7           | male   |



# What a data frame looks like to R

penguins



0x136dd5608

00000000  
01111000  
00110001  
00110011  
00110110  
01100100  
01100100  
00110101  
00110110  
00000000  
00111000

type

length

data

names

list

4

|             |             |             |             |
|-------------|-------------|-------------|-------------|
| 0x120a6d600 | 0x120c37400 | 0x120c2ba00 | 0x136aa9a00 |
|-------------|-------------|-------------|-------------|

| species | island | bill_length_mm | sex |
|---------|--------|----------------|-----|
|---------|--------|----------------|-----|

0010111

00000100

00000000 etc. etc. etc.  
01111000  
00110001  
00110010  
etc.

01110011 01101001 etc. etc.  
01110000 01110011  
01100101 01101100  
01100011 01101100  
01101001 01101110  
01100101 01100100  
01110011

type

length

data

char

4

|        |        |        |        |
|--------|--------|--------|--------|
| Adelie | Adelie | Adelie | Adelie |
|--------|--------|--------|--------|

01100011

00000100

01000001 01000001 01000001 01000001  
01100100 01100100 01100100 01100100  
01100101 01100101 01100101 01100101  
01101100 01101100 01101100 01101100  
01101001 01101001 01101001 01101001  
01100101 01100101 01100101 01100101

# Converting between data types

It is often possible and useful to convert between data types (called **coercion** in R)

Must be a homogenous data type (vector, matrix, or column in a data frame)

Must make logical sense (e.g., “2” can be coerced to integer but “kangaroo” cannot)

To coerce, use a function: `as.integer()`, `as.logical()`, `as.character()`, etc.

Coercion rules to be aware of:

|                      |                                                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| Numeric to integer   | truncates any decimal values (does not round!)                                           |
| Numeric to logical   | <code>0</code> becomes <code>FALSE</code> ; non-zero values become <code>TRUE</code>     |
| Logical to numeric   | <code>TRUE</code> becomes <code>1</code> , <code>FALSE</code> becomes <code>0</code>     |
| Numeric to character | numerals and symbols become characters                                                   |
| Character to numeric | must be a formatted number ( <code>-</code> , <code>+</code> and <code>.</code> allowed) |

And more; experiment and check documentation to avoid unexpected results!



# Missing values

R provides three special values that represent missing, invalid, or undefined information

`NA` a missing value; acronym for not available

`NaN` an invalid mathematical result (e.g.,  $1/0$ ); acronym for not a number

`NULL` a value that is undefined (e.g. vector of length 0)

Points to remember:

Do not use quotes: `"NA"` is interpreted a character value

Do not use in mathematical operations: `my_var + NA` substitutes every item with `NA`

Do not use in logical tests: `my_var == NA` returns `NA`

To identify missing values:

`is.na(my_vec)` returns a logical vector with NAs FALSE, all others TRUE

`which(is.na(my_vec))` returns the position(s) of any NAs in the vector

